Project 2 (README.pdf)

Graded

2 Days, 20 Hours Late

Student

Colin Skinner

Total Points

3 / 3 pts

Question 1

Description of strategy

1 / 1 pt

- ✔ **+ 0.33 pts** Strategy for small policy
- ✔ **+ 0.33 pts** Strategy for medium policy
- ✔ **+ 0.34 pts** Strategy for large policy

+ 0 pts Description not provided

Question 2

Performance Characteristics

1 / 1 pt

- ✔ **+ 0.33 pts** Running/training time of small policy
- ✔ **+ 0.33 pts** Running/training time of medium policy
- ✔ **+ 0.34 pts** Running/training time of large policy

+ 0 pts Runtimes not provided

Question 3

Code

1 / 1 pt

- ✔ **– 0 pts** Correct

– 1 pt Missing code

Questions assigned to the following page: 1 and 2

Project 2: Reinforcement Learning

Colin Skinner colin.skinner@stanford.edu
AA228/CS238, Stanford University

1. Algorithm Description

The script first loads the dataset and creates an estimate of the MDP model using MaximumLikelihoodMDP, by iterating over the observed transitions multiple times to update counts and rewards. Then it uses SoftmaxExploration to make the policy that selects actions by balancing exploration of different actions with exploting them based on their learned values from the SoftmaxExploration.

2. Algorithm Runtime

For all cases, the epoch MaximumLikelihoodMDP training time seemed to be the most time-consuming because it is actually forming its MDP model.

2.1 small.csv

Running with 1000 epochs yields about 15 seconds of runtime.

```
real    0m14.818s
user    0m14.113s
sys     0m0.311s
```

2.2 medium.csv

Running with 1000 epochs yields about 46 seconds of runtime.

Unix *time* command output:

```
real    0m46.738s
user    0m45.452s
sys     0m0.573s
```

2.3 large.csv

Running with 1000 epochs yields about 28 seconds of runtime.

Unix *time* command output:

```
real    0m28.603s
user    0m28.649s
sys     0m0.334s
```

Question assigned to the following page: 3

3. Code

I split the code into a file containing all the functions (*policies.jl*) and a script that loads the cache, calls the functions, and saves the outputs (*project2.jl*).

3.1 project2.jl

```
include("policies.jl")

save_outputs = true

function train_max_likelihood(name, csv_name, cache_name, save_name, rows,
    cols, rate, discount = 0.95, iters = 1000)

    max_likelihood = MaximumLikelihoodMDP(1:rows, 1:cols,
        Dict{Tuple{Int,Int}, Dict{Int,Int}}(),  # Can't fit in a 3D array
        Dict{Tuple{Int,Int}, Float64}(),        # rewards also in a dictionary
        discount,
        zeros(Float64, rows)
    )

    lines = get_lines(csv_name)

    print("Training "); println(name)
    for i in 1:iters
        for line in lines
            s, a, r, s = line
            update!(max_likelihood, s, a, r, s)
        end
    end
    println("Done")

    softmax = SoftmaxExploration(.05, 0.99)
    println("Solved")

    softmax_policy(state::Int) = softmax(max_likelihood, state)

    save_policy(save_name, softmax_policy, rows)
    save_action_value_function(cache_name, max_likelihood)

    print("Saved "); println(name)
end

##################################################
#   Small CSV - 100 states
##################################################

# train_max_likelihood("small", "small.csv", "small_cache", "policies/small.
    policy", 100, 4, 1, 1000)
```

Question assigned to the following page: 3


```
# train_max_likelihood("medium", "medium.csv", "medium_cache", "policies/
    medium.policy", 50000, 7, 1, 1000)
train_max_likelihood("large", "large.csv", "large_cache", "policies/large.
    policy", 302020, 9, 0.95, 1000)
```

3.2 policies.jl

```
using Printf
using Serialization
using Distributions
using StatsBase

function save_policy(filename::String, policy_func::Function, num_states::Int
    )
    open(filename, "w") do f
        for si in 1:num_states
            write(f, @sprintf("%d\n", policy_func(si)))
        end
    end
end

function save_action_value_function(filename::String, model)
    serialize(filename, model)
end

function load_action_value_function(filename::String)
    return deserialize(filename)
end

function get_lines(filename::String)
    lines = Vector{Vector{Int}}()
    open(filename, "r") do input
        header = readline(input) # Header is ignored

        # Parse lines
        for line in eachline(input)
            sample = parse.(Int, split(line, ','))
            push!(lines, sample)
        end
    end
    return lines
end

################################################
#   Q Learning
################################################

mutable struct QLearning
     # state space (assumes 1:nstates)
```

Question assigned to the following page: 3


```
     # action space (assumes 1:nactions)
     # discount
    Q # action value function
     # learning rate
end

lookahead(model::QLearning, s, a) = model.Q[s,a]

function update!(model::QLearning, s, a, r, s)
    , Q,  = model., model.Q, model.
    Q[s,a] += *(r + *maximum(Q[s,:]) - Q[s,a])
    return model
end

################################################
#   MaximumLikelihoodMDP
################################################

mutable struct MaximumLikelihoodMDP
     # state space (assumes 1:nstates)
     # action space (assumes 1:nactions)
    N # transition count N(s,a,s) - dictionary based
     # reward sum (s, a) - dictionary based
     # discount
    U # value function

end

function lookahead(model::MaximumLikelihoodMDP, s, a)
    , U,  = model., model.U, model.
    key = (s, a)

    state_dict = get(model.N, key, Dict())
    if isempty(state_dict)
        return 0.0
    end

    n = sum(values(state_dict))
    r = model.[key] / n

    return r +  * sum((count/n) * U[s] for (s, count) in state_dict)
end

function backup(model::MaximumLikelihoodMDP, U, s)
    vals = [lookahead(model, s, a) for a in model.]
    return isempty(vals) ? 0.0 : maximum(vals)
end

function update!(model::MaximumLikelihoodMDP, s, a, r, s)
    key = (s, a)
```

Question assigned to the following page: 3


```
    state_dict = get(model.N, key, Dict{Int,Int}())
    state_dict[s] = get(state_dict, s, 0) + 1

    model.N[key] = state_dict
    model.[key] = get(model., key, 0.0) + r

    return model
end

################################################
#   ValueFunctionPolicy
################################################

struct ValueFunctionPolicy
     # problem
    U # utility function
end

function greedy(::MaximumLikelihoodMDP, U::Vector{Float64}, s::Int)
    u, a = findmax(a -> lookahead(, s, a), .)
    return (a=a, u=u)
end

(::ValueFunctionPolicy)(s) = greedy(., .U, s).a

################################################
#   ValueIteration
################################################

struct ValueIteration
    k_max # maximum number of iterations
end

function solve(M::ValueIteration, ::MaximumLikelihoodMDP)
    U = [0.0 for s in .]
    for k = 1:M.k_max
        U = [backup(, U, s) for s in .]
    end
    # Update the model's value function
    .U = U
    return ValueFunctionPolicy(, U)
end

################################################
#   Softmax
################################################

mutable struct SoftmaxExploration
     # precision parameter
```

Question assigned to the following page: 3


```
     # precision factor
end

function normalize(weights)
    total = sum(weights)
    return weights ./ total
end

function (::SoftmaxExploration)(model::MaximumLikelihoodMDP, s)
    # Q-values for actions
    Q_values = [lookahead(model, s, a) for a in model.]

    Q_shift = Q_values .- maximum(Q_values)
    weights = exp.(. * Q_shift)
    . *= .

    # return s sampled action
    return rand(Categorical(normalize(weights)))
end
```